NRW Air Quality: A Serverless Data Pipeline on AWS

Ingesting, cleaning and analysing 2.5 million air-quality readings with S3, Athena and Tableau

Mahmoud Khatab · ORCID 0009-0003-6416-1006 · November 2026

Overview

In this project I built an end-to-end, serverless data pipeline on Amazon Web Services for open air-quality data covering monitoring stations in North Rhine-Westphalia (NRW), Germany. I ingested raw measurements from the public OpenAQ open-data archive into Amazon S3, queried them in place with Amazon Athena (SQL), summarised them, and visualised the results in an interactive Tableau dashboard. I kept the scope deliberately as a **data-engineering and analytics demonstration** — building a reproducible pipeline and descriptive analysis.

Data source

- **Provider:** OpenAQ — a non-profit aggregator of global, ground-level air-quality data (openaq.org).
- **Access route:** the OpenAQ public open-data archive on AWS (`s3://openaq-data-archive` , us-east-1), gzipped CSV, partitioned by location / year / month.
- **Scope:** a sample of **~27 NRW monitoring stations** (filtered from the German set by an approximate lat/lon bounding box), yielding **2,535,748 hourly readings** across six pollutants (NO_2, PM10, PM2.5, O_3, NO, SO_2).

Architecture

```
OpenAQ public S3 archive (us-east-1, gzipped CSV)
        │   Python + boto3 — anonymous (unsigned) read,
        │   credentialed write; streamed through RAM (no local disk)
        ▼
Amazon S3  —  s3://mahmoud-khatab-openaq-data-2026/raw-archive/
        │   Amazon Athena external table (OpenCSVSerde)
        ▼
SQL aggregation in Athena  →  summary CSV (~8,372 rows)
        ▼
Tableau Public  —  interactive dashboard (trend · map · ranking)
```

Method

1. **Catalogue:** I fetched the OpenAQ location catalogue via the v3 REST API (paginated), then filtered it to NRW stations by bounding box.
2. **Ingestion:** I copied each station's archive files from the public bucket into my own private S3 bucket. Because the public bucket allows anonymous reads but denies signed reads, my script uses an *unsigned* client to read and a credentialed client to write, streaming each file through memory so my laptop storage was not a constraint.

3. **Table definition:** I defined an Athena external table over the raw files.
4. **Cleaning:** see below.
5. **Analysis & export:** I aggregated the data with SQL and exported a compact summary CSV for visualisation.

Design decision: handling inconsistent quoting

My initial table used a plain delimited reader, and it produced duplicated categories — `"pm10"` and `pm10` were counted as different pollutants — because the archive files are inconsistently quoted (some rows wrap fields in double quotes, some do not). I fixed this by redefining the table with the quote-aware `OpenCSVSerde`, which strips quotes uniformly. Since this SerDe reads every column as text, I convert numeric columns at query time with `TRY_CAST` (which safely ignores malformed values), and I apply a `value BETWEEN 0 AND 2000` filter to remove negative readings and implausible spikes.

Findings

1. Dataset scale. 2,535,748 readings across the sampled NRW stations.

2. Readings per pollutant.

Pollutant	Readings
NO_2	751,774
PM10	703,999
PM2.5	492,703
O_3	385,825
NO	109,707
SO_2	91,740

3. Average and peak concentration per pollutant (µg/m³).

Pollutant	Average	Maximum
O_3	48.67	298.71
NO_2	19.73	209.25
PM10	15.62	1560.46
PM2.5	9.30	1083.50
NO	7.30	643.74
SO_2	3.25	273.73

Ozone shows the highest average, consistent with its regional, photochemical nature; NO_2 reflects traffic-related emissions. Very high maxima (e.g. PM10 at 1560) are isolated spikes or sensor artefacts rather than typical levels.

Interactive dashboard

I built a Tableau Public dashboard with three linked views — a monthly trend per pollutant, a map of stations coloured by concentration, and a ranking of the most-polluted stations — tied together by a shared pollutant filter:

https://public.tableau.com/app/profile/mahmoud.khatab/viz/NRW_AirQuality/Dashboard1

Reproducibility

The files I include reproduce my full pipeline:

- `get_ids.py` — extract OpenAQ location IDs by country from the catalogue.
- `get_nrw_ids.py` — filter German stations to NRW by bounding box.
- `copy_archive.py` — stream station files from the public archive into S3.
- `01_create_table.sql` — Athena database and external table (OpenCSVSerde).
- `02_analysis_queries.sql` — the counts, per-pollutant statistics and summary export.

Tools

Amazon S3, Amazon Athena (Trino SQL), Python (boto3), AWS CLI, Tableau Public. Data © OpenAQ contributors, used under the OpenAQ terms of use.